

Angular Development Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This Angular course is designed to take learners from the fundamentals of TypeScript and component-based frontend development to building production-ready, scalable enterprise applications using modern Angular.

The syllabus covers TypeScript, Angular CLI, components, templates, data binding, directives, pipes, dependency injection, services, signals, forms, routing, HTTP APIs, RxJS, state management, authentication, testing, performance, SSR, accessibility, security, deployment, and professional project development.

The course follows a **modern standalone-first Angular approach** while also teaching **NgModules and legacy patterns** so learners can confidently work on both new and existing enterprise applications.

1. Prerequisites

Before starting Angular, learners should have a working understanding of:

HTML

- HTML document structure
- Semantic HTML
- Forms and input elements
- Tables
- Lists
- Links and navigation
- Images and multimedia
- HTML5 semantic elements
- Accessibility basics

CSS

- CSS selectors
- Box model
- Display properties
- Positioning
- Flexbox
- CSS Grid
- Responsive design
- Media queries
- Pseudo-classes and pseudo-elements
- CSS variables
- Transitions and animations

JavaScript Fundamentals

- Variables
- Data types
- Operators
- Conditions
- Loops
- Functions
- Arrays
- Objects
- DOM basics
- Events

- Scope
- Closures
- Error handling

2. Modern JavaScript for Angular

ES6+ Features

- let and const
- Arrow functions
- Template literals
- Default parameters
- Rest parameters
- Spread operator
- Destructuring
- Object shorthand
- Optional chaining
- Nullish coalescing
- Modules
- Classes
- Promises
- Async/await

Array Methods

- map()
- filter()
- reduce()
- find()
- findIndex()
- some()
- every()
- forEach()
- sort()

Asynchronous JavaScript

- Callbacks
- Promises
- Promise chaining
- async
- await
- Fetch concepts
- Error handling

3. TypeScript Fundamentals

Angular development is strongly based on TypeScript.

Introduction to TypeScript

- What is TypeScript?
- Why TypeScript?
- TypeScript vs JavaScript
- TypeScript compilation
- Static typing

Basic Types

- string
- number
- boolean
- null
- undefined
- unknown
- any
- never
- void

Arrays and Tuples

- Typed arrays
- Readonly arrays
- Tuples

Objects

- Object types
- Optional properties
- Readonly properties

Functions

- Parameter types
- Return types
- Optional parameters
- Default parameters
- Rest parameters
- Function types

4. Advanced TypeScript

Interfaces

- Creating interfaces
- Optional properties
- Readonly properties
- Extending interfaces

Type Aliases

- Object types
- Union types
- Intersection types

Enums

- Numeric enums
- String enums
- Alternatives to enums

Classes

- Constructors
- Properties
- Methods
- Access modifiers
- Inheritance
- Abstract classes
- Static members

Generics

- Generic functions
- Generic interfaces
- Generic classes
- Constraints

Utility Types

- Partial
- Required
- Readonly
- Pick
- Omit
- Record

Type Narrowing

- typeof
- instanceof
- in
- Type guards
- Discriminated unions

5. Introduction to Angular**Angular Fundamentals**

- What is Angular?
- Why Angular?
- Angular ecosystem
- Angular as a frontend framework
- Single Page Applications
- Component-based architecture

Angular Features

- Components
- Templates
- Dependency injection
- Routing
- Forms
- HTTP client
- RxJS integration
- Signals
- Testing support
- SSR support

Angular vs Other Frameworks

- Angular vs React
- Angular vs Vue
- Opinionated vs flexible frameworks
- Enterprise application suitability

6. Angular Development Environment**Required Tools**

- Node.js
- npm

- Angular CLI
- VS Code
- Git
- GitHub
- Browser developer tools

Angular CLI Installation

- Installing Angular CLI
- Checking CLI version
- Updating Angular CLI

Creating Angular Applications

- `ng new`
- Project options
- Routing option
- Stylesheet options
- Standalone application structure

Running Applications

- `ng serve`
- Development server
- Hot reload
- Production build

7. Angular CLI

Essential CLI Commands

- `ng new`
- `ng serve`
- `ng build`
- `ng generate`
- `ng test`
- `ng update`

Code Generation

- Components
- Services
- Directives
- Pipes
- Guards
- Interceptors
- Interfaces
- Classes

CLI Configuration

- Workspace configuration
- Build targets
- Environment settings

8. Angular Project Structure

Important Files

- `package.json`
- `angular.json`

- tsconfig.json
- main.ts
- index.html
- Application configuration

Source Folder

- Components
- Services
- Models
- Routes
- Assets
- Styles

Recommended Folder Organization

- core
- shared
- features
- services
- models
- guards
- interceptors
- utils

9. Angular Components

Component Fundamentals

- What is a component?
- Component decorator
- Selector
- Template
- Styles
- Component class

Creating Components

- Angular CLI
- Manual creation

Component Metadata

- selector
- template
- templateUrl
- styles
- styleUrls
- imports
- providers

Standalone Components

- Standalone-first development
- Importing dependencies directly
- Component-level dependency declarations

10. Angular Templates

Template Syntax

- HTML in Angular
- Template expressions
- Template statements

Interpolation

- Displaying component data
- Expressions inside interpolation

Template Variables

- Local template references
- Accessing DOM elements

Safe Template Practices

- Avoiding excessive logic
- Keeping templates readable

11. Data Binding**One-Way Binding**

- Interpolation
- Property binding
- Attribute binding
- Class binding
- Style binding

Event Binding

- Click events
- Change events
- Input events
- Keyboard events

Two-Way Binding

- [(ngModel)]
- FormsModule
- Two-way binding concepts

Modern Model Binding

- Signal-based model communication concepts

12. Input and Output Communication**Parent-to-Child Communication**

- Component inputs
- Passing primitive values
- Passing objects
- Passing arrays

Child-to-Parent Communication

- Component outputs
- Event emission

Modern Signal-Based Component APIs

- Signal inputs
- Signal outputs

- Model inputs
- Required inputs
- Input transformations

Component Communication Best Practices

- Avoiding tight coupling
- Designing clear component APIs

13. Directives

Directive Fundamentals

- What is a directive?
- Component directives
- Attribute directives
- Structural directives

Built-In Attribute Directives

- `ngClass`
- `ngStyle`

Legacy Structural Directives

- `*ngIf`
- `*ngFor`
- `*ngSwitch`

Custom Attribute Directives

- Creating directives
- Host interaction
- Input properties

14. Modern Template Control Flow

Conditional Rendering

- `@if`
- `@else`
- `@else if`

Repetition

- `@for`
- Tracking list items
- Empty states

Switch Control

- `@switch`
- `@case`
- `@default`

Benefits

- Cleaner templates
- Better readability
- Improved control-flow semantics

15. Deferred Loading in Templates

`@defer`

- Deferring expensive UI
- Lazy rendering

Defer Triggers

- Viewport
- Interaction
- Idle
- Timer concepts

Placeholder Blocks

- @placeholder

Loading Blocks

- @loading

Error Blocks

- @error

Use Cases

- Dashboards
- Heavy charts
- Large feature components

16. Pipes

Built-In Pipes

- Date pipe
- Currency pipe
- Decimal pipe
- Percent pipe
- Uppercase pipe
- Lowercase pipe
- Title case pipe
- JSON pipe

Async Pipe

- Working with Observables
- Automatic subscription management

Custom Pipes

- Creating a pipe
- Transform method
- Pipe arguments

Pure and Impure Pipes

- Performance implications

17. Angular Signals

Introduction to Signals

- Reactive state concepts
- Why signals?
- Writable signals

Signal Operations

- Creating signals
- Reading signal values
- Updating signals
- Setting values

Computed Signals

- Derived state
- Memoized calculations

Effects

- Reacting to signal changes
- Appropriate effect usage

Signal Best Practices

- Derived state
- Avoiding unnecessary effects
- Local UI state

18. Signals and RxJS Interoperability

Signals vs Observables

- State vs event streams
- Synchronous vs asynchronous reactivity

Converting Observables to Signals

- Observable-to-signal concepts

Converting Signals to Observables

- Signal-to-observable concepts

Choosing the Right Tool

- Component state
- API streams
- Events
- Server state

19. Component Lifecycle

Lifecycle Concepts

- Component creation
- Rendering
- Change detection
- Destruction

Lifecycle Hooks

- `ngOnInit`
- `ngOnChanges`
- `ngDoCheck`
- `ngAfterContentInit`
- `ngAfterContentChecked`
- `ngAfterViewInit`
- `ngAfterViewChecked`
- `ngOnDestroy`

Modern Render Hooks

- Render-phase concepts
- Post-render operations

Lifecycle Best Practices

- Cleanup
- Avoiding expensive work

20. Dependency Injection

Dependency Injection Fundamentals

- What is DI?
- Benefits of DI
- Inversion of control

Angular Injector

- Providers
- Tokens
- Injection hierarchy

Injecting Dependencies

- Constructor injection
- `inject()` function

Provider Configuration

- Class providers
- Value providers
- Factory providers
- Existing providers

Injection Tokens

- Custom tokens
- Configuration objects

21. Angular Services

Services

- Creating services
- Business logic separation
- Reusable functionality

Service Registration

- Root-level services
- Component-level providers
- Route-level providers

Common Service Use Cases

- API clients
- Authentication
- Logging
- Configuration
- Shared state

Service Design Best Practices

- Single responsibility
- Testability

- Reusability

22. RxJS Fundamentals

Reactive Programming

- What is reactive programming?
- Observable streams

Core RxJS Concepts

- Observable
- Observer
- Subscription
- Operators

Creating Observables

- of
- from
- Events
- HTTP streams

Subscription Management

- Subscribe
- Unsubscribe
- Automatic cleanup patterns

23. RxJS Operators

Transformation Operators

- map
- switchMap
- mergeMap
- concatMap
- exhaustMap

Filtering Operators

- filter
- take
- takeUntil
- distinctUntilChanged
- debounceTime

Combination Operators

- combineLatest
- forkJoin
- merge
- concat

Utility Operators

- tap
- finalize
- delay

Error Handling

- catchError
- retry

24. Subjects

Subject

- Multicasting
- Manual value emission

BehaviorSubject

- Current value
- Shared application state

ReplaySubject

- Replaying values

AsyncSubject

- Final-value behavior

Subject vs Signal

- When each is appropriate

25. Subscription Management

Problems with Manual Subscriptions

- Memory leaks
- Cleanup complexity

Recommended Techniques

- Async pipe
- Framework-managed lifecycle
- `takeUntilDestroyed`
- Signals where appropriate

Avoiding Nested Subscriptions

- Using higher-order RxJS operators

26. Template-Driven Forms

FormsModule

- Setup
- Form directives

ngModel

- Two-way binding
- Field state

Validation

- Required
- Min length
- Max length
- Pattern
- Email

Form State

- Valid

- Invalid
- Dirty
- Pristine
- Touched
- Untouched

Error Messages

- Conditional feedback

27. Reactive Forms

ReactiveFormsModule

- Setup

FormControl

- Creating controls
- Reading values
- Updating values

FormGroup

- Grouping controls
- Nested groups

FormArray

- Dynamic forms
- Repeating controls

FormBuilder

- Simplifying form creation

Typed Reactive Forms

- Strong typing
- Non-nullable controls

28. Form Validation

Built-In Validators

- Required
- Min
- Max
- Min length
- Max length
- Pattern
- Email

Custom Validators

- Creating validation functions

Cross-Field Validation

- Password confirmation
- Date range validation

Async Validation

- Server-side uniqueness checks

Dynamic Validation

- Adding validators
- Removing validators
- Conditional validation

29. Signal-Based Forms**Signal Form Concepts**

- Form models with signals
- Field trees
- Reactive field state

Binding Fields

- Signal-based field binding

Schema-Based Validation

- Validation rules
- Error state

Form State with Signals

- Validity
- Required state
- Disabled state
- Error collections

When to Use

- New signal-centric applications
- Comparing with reactive forms

30. Angular Routing**Router Fundamentals**

- Single Page Application routing
- Route configuration

Defining Routes

- Path
- Component
- Redirects
- Wildcards

Router Outlet

- Rendering routed components

Navigation

- routerLink
- Programmatic navigation

Active Routes

- Navigation styling

31. Route Parameters**Path Parameters**

- Dynamic URLs

- Product/user IDs

ActivatedRoute

- Reading parameters

Query Parameters

- Search
- Filters
- Pagination

URL Fragments

- Anchor navigation

Param Maps

- Safe parameter access

32. Nested and Child Routes

Child Routes

- Nested screens
- Feature layouts

Multiple Outlets

- Named outlets concepts

Route Hierarchies

- Parent routes
- Child routes

Route-Level Layouts

- Dashboard layouts
- Admin layouts

33. Route Guards

Authentication Guards

- Protecting authenticated routes

Authorization Guards

- Role-based access

Other Guard Concepts

- Activation checks
- Deactivation checks
- Match checks

Functional Guards

- Modern functional guard style

Guard Best Practices

- Frontend protection vs backend authorization

34. Route Resolvers and Route Data

Static Route Data

- Metadata
- Titles
- Breadcrumb data

Resolvers

- Loading route data before activation

When to Use Resolvers

- Detail pages
- Required initial data

Loading UX Considerations

35. Lazy Loading

Lazy-Loaded Routes

- Loading features on demand

Lazy Components

- Dynamic import concepts

Benefits

- Smaller initial bundle
- Faster first load

Preloading Strategies

- Preload all
- Custom strategies

Route-Level Code Splitting

36. HTTP Client

HttpClient

- Making API calls

HTTP Methods

- GET
- POST
- PUT
- PATCH
- DELETE

Request Configuration

- Headers
- Parameters
- Response types

Typed Responses

- Interfaces
- DTO models

Error Handling

- HTTP errors
- User-friendly messages

37. API Service Layer

Creating API Services

- Centralized service design

REST API Integration

- CRUD operations

API Models

- Request DTOs
- Response DTOs

Environment-Based URLs

- Development
- Staging
- Production

Service-Layer Best Practices

- Separation of concerns
- Error strategy
- Reusability

38. HTTP Interceptors

Interceptor Concepts

- Request interception
- Response interception

Common Use Cases

- Authentication tokens
- Request headers
- Global error handling
- Logging
- Loading indicators

Functional Interceptors

- Modern interceptor style

Interceptor Ordering

39. Authentication

Authentication Concepts

- Login
- Logout
- Registration
- User session

Token-Based Authentication

- Access tokens
- Refresh tokens
- JWT concepts

Authentication Service

- Current-user state

- Token handling

Route Protection

- Authentication guards

Session Restoration

- Application startup

40. Authorization

Role-Based Access Control

- User
- Manager
- Administrator

Permission-Based Access

- Feature permissions
- Action permissions

UI Authorization

- Hiding/showing controls

Route Authorization

- Guard-based access

Important Security Principle

- Backend must enforce authorization

41. Browser Storage

Local Storage

- Reading
- Writing
- Removing values

Session Storage

Cookies

- Basic concepts

Storage Security

- Sensitive data risks
- XSS considerations
- Token-storage considerations

42. State Management Fundamentals

State Categories

- Local component state
- Shared feature state
- Application state
- Server state

State with Signals

- Local and shared signal state

State with Services

- Injectable stores
- Encapsulated state

RxJS-Based State

- BehaviorSubject stores

Choosing a State Strategy

- Simplicity vs complexity
- Team size
- Application size

43. NgRx**NgRx Fundamentals**

- Store
- State
- Actions
- Reducers
- Selectors
- Effects

Actions

- Event-driven state changes

Reducers

- Immutable updates

Selectors

- Derived state

Effects

- API calls
- Async workflows

Entity Concepts

- Normalized collections

NgRx DevTools

- State inspection
- Time-travel debugging concepts

44. Modern Signal-Based State Management**Signal Stores**

- Signal-based application state concepts

Computed State

- Derived state

Store Methods

- Updating state

Async Operations

- Loading

- Success
- Error states

Choosing Between

- Services + signals
- NgRx
- Lightweight signal stores

45. Change Detection

Change Detection Fundamentals

- How Angular updates the UI

Default Strategy

OnPush Strategy

- Immutable input patterns
- Performance benefits

Signals and Change Detection

- Fine-grained reactive updates

Zoneless Angular Concepts

- Reducing Zone.js dependency
- Explicit reactive state
- Modern change-detection direction

46. Performance Optimization

Rendering Performance

- Avoiding expensive template expressions
- Stable tracking in loops

Change Detection Optimization

- OnPush
- Signals
- Immutable data

Lazy Loading

- Features
- Components

Deferred Loading

- @defer

RxJS Optimization

- Debouncing
- Cancellation
- Avoiding duplicate requests

Bundle Optimization

- Tree shaking
- Code splitting
- Bundle analysis

47. Angular Resource and Async Data Patterns

Resource-Oriented Data Loading

- Reactive request state
- Loading
- Success
- Error

HTTP Resource Concepts

- Signal-friendly HTTP data access

RxJS Resource Interoperability

- Combining signal and Observable workflows

When to Use

- Component-oriented async state
- API-driven screens

48. Content Projection

ng-content

- Projecting content

Single-Slot Projection

Multi-Slot Projection

Selectors

- Named content regions

Use Cases

- Cards
- Dialogs
- Layout components
- Reusable containers

49. View and Content Queries

View Queries

- Accessing child components
- Accessing DOM elements

Content Queries

- Projected children

Query Types

- Single child
- Multiple children

Signal-Based Query APIs

- Reactive query concepts

50. Dynamic Components

Dynamic Rendering

- Runtime component selection

ViewContainerRef Concepts

Programmatic Component Creation

Common Use Cases

- Modal systems
- Dynamic dashboards
- Plugin-like UI
- Config-driven forms

51. Custom Directives and Advanced Template APIs

Attribute Directives

- Host behavior
- Inputs

Host Bindings

- Managing host properties

Host Listeners

- Reacting to DOM events

Reusable Behavior

- Tooltip behavior
- Permissions
- Input formatting

52. Angular Animations and UI Motion

CSS-Based Animation

- Transitions
- Keyframes

Angular Animation Concepts

- Enter/leave transitions
- State transitions

Modern UI Motion

- Native CSS-first approaches
- View-transition concepts

Accessibility

- Reduced-motion preferences

53. Styling Angular Applications

Component Styles

- Scoped styles
- Global styles

CSS

- SCSS
- Sass variables
- Mixins

Utility CSS

- Tailwind CSS concepts

UI Libraries

- Angular Material
- PrimeNG
- Bootstrap

Design Systems

- Theme tokens
- Reusable UI components

54. Angular Material

Material Setup

- Installing Angular Material
- Theme configuration

Common Components

- Buttons
- Inputs
- Cards
- Dialogs
- Tables
- Selects
- Date pickers
- Tabs
- Menus
- Sidenav

Material CDK Concepts

- Overlay
- Accessibility
- Drag and drop
- Virtual scrolling

55. Tables and Data-Heavy UI

Data Tables

- Displaying collections
- Columns

Sorting

Filtering

Pagination

Server-Side Data Tables

- API pagination
- Sorting parameters
- Filter parameters

Large Data Sets

- Virtual scrolling

56. Error Handling

JavaScript Errors

- Try/catch

HTTP Errors

- Status codes
- API messages

Global Error Handling

- Centralized error strategy

User Experience

- Friendly messages
- Retry options
- Error pages

Logging

- Client-side logging concepts

57. Testing Fundamentals

Testing Types

- Unit testing
- Integration testing
- End-to-end testing

Angular Testing Utilities

- TestBed
- Component fixtures
- Dependency injection in tests

Modern Test Runner Concepts

- Vitest
- Assertions
- Mocks
- Spies

Test Organization

- Arrange
- Act
- Assert

58. Component Testing

Creating Component Tests

- Component rendering
- Change detection

Testing Inputs

Testing Outputs

DOM Queries

User Interaction

- Clicks
- Inputs
- Form submission

Testing Signal State

Testing Conditional Rendering

59. Service and HTTP Testing

Testing Services

- Dependency injection
- Mock dependencies

HTTP Testing

- Mock HTTP backend
- Request expectations
- Mock responses

Testing Error Cases

Testing Interceptors

Testing Async Logic

60. Routing Testing

Testing Navigation

Router Testing Harness

Testing Route Parameters

Testing Guards

Testing Protected Routes

Testing Redirects

61. End-to-End Testing

E2E Concepts

- User workflows

Common Tools

- Playwright
- Cypress

Typical Test Scenarios

- Login
- Search
- Form submission
- CRUD operations
- Protected routes

E2E Best Practices

- Stable selectors
- Independent tests

- Test data strategy

62. Accessibility

Accessibility Fundamentals

- Semantic HTML
- Keyboard navigation
- Focus management
- Labels

ARIA

- Roles
- States
- Properties

Accessible Angular Components

- Forms
- Dialogs
- Menus
- Navigation

Angular CDK Accessibility

- Focus utilities
- Live announcer concepts

Accessibility Testing

63. Security

XSS Protection

- Template escaping
- Sanitization

Dangerous HTML

- Inner HTML risks

Authentication Security

Authorization Security

CSRF Concepts

Content Security Policy Concepts

Dependency Security

- npm audit
- Updating packages

Frontend Secrets

- Why secrets must not be stored in frontend code

64. Environment Management

Environment Configuration

- Development
- Testing

- Staging
- Production

API Base URLs**Build-Time Configuration****Runtime Configuration Concepts****Secure Configuration Practices****65. Server-Side Rendering****SSR Fundamentals**

- Client-side rendering vs server-side rendering

Angular SSR

- Server rendering concepts

Benefits

- Initial load performance
- SEO
- Social sharing

SSR Considerations

- Browser-only APIs
- Server-safe code
- Authentication

66. Hydration**Hydration Fundamentals**

- Reusing server-rendered HTML

Client Hydration**Incremental Hydration Concepts****Event Replay Concepts****Performance Benefits****SSR + Hydration Architecture****67. SEO for Angular Applications****SPA SEO Challenges****Metadata**

- Page titles
- Meta descriptions

Router-Based Metadata**SSR for SEO****Structured Data Concepts****Canonical URLs**

Sitemap and Robots Concepts

68. Progressive Web Application Concepts

PWA Fundamentals

- Installable applications
- Offline experiences

Service Workers

Web App Manifest

Caching Strategies

Update Strategy

Push Notification Concepts

69. Internationalization

i18n Fundamentals

- Multiple languages
- Locale support

Translation Concepts

Locale-Sensitive Formatting

- Dates
- Currency
- Numbers

Translation Workflow

Runtime Translation Library Concepts

70. Angular Application Architecture

Layered Architecture

- Presentation
- Feature/business layer
- Data access
- Shared utilities

Feature-Based Structure

Core Services

Shared Components

Domain Models

Separation of Concerns

Scalable Enterprise Architecture

71. Standalone-First Architecture

Standalone Components

- Components without feature NgModules

Standalone Directives**Standalone Pipes****Application Configuration****Provider Functions****Route-Level Dependencies****Benefits**

- Simpler dependency structure
- Better tree shaking
- Easier lazy loading

72. NgModules and Legacy Angular Applications

Modern Angular applications are standalone-first, but enterprise developers must still understand NgModules.

NgModule Fundamentals

- @NgModule
- Declarations
- Imports
- Providers
- Exports
- Bootstrap

Feature Modules**Shared Modules****Core Module Pattern****Lazy-Loaded Modules****Migrating Toward Standalone APIs**

73. Micro Frontends

Micro Frontend Concepts

- Independently developed frontend features
- Independent deployment

Benefits and Tradeoffs**Module Federation Concepts****Shared Dependencies****Routing Between Applications****Enterprise Use Cases**

74. Monorepos and Nx

Monorepo Concepts**Nx Introduction**

- Workspace structure
- Applications

- Libraries

Shared Libraries

- UI libraries
- Data-access libraries
- Feature libraries

Dependency Boundaries

Build Caching Concepts

Enterprise Development Workflows

75. Git and GitHub for Angular Developers

Git Fundamentals

- Repository
- Clone
- Add
- Commit
- Push
- Pull

Branching

- Feature branches
- Bug-fix branches
- Release branches

Pull Requests

Code Reviews

Merge Conflicts

GitHub Portfolio

- Professional repositories
- README
- Screenshots
- Live demo links

76. Code Quality

ESLint

- Linting rules
- Angular-specific linting

Prettier

- Formatting

TypeScript Strict Mode

- Type safety

Naming Conventions

- Components
- Services
- Guards
- Directives

- Pipes

Clean Code

- Single responsibility
- Small methods
- Reusable components
- Avoiding duplication

77. Debugging

Browser Developer Tools

- Console
- Network
- Elements
- Application

Angular DevTools

- Component inspection
- Dependency inspection
- Performance profiling

RxJS Debugging

Common Angular Errors

- Dependency injection errors
- Change detection errors
- Template errors
- Routing errors
- Observable subscription issues

78. Production Build and Deployment

Production Builds

- Build command
- Build optimization

Deployment Options

- Azure Static Web Apps
- AWS
- Firebase Hosting
- Netlify
- Vercel
- Nginx
- Apache

Environment Variables

SPA Route Configuration

- Server fallback rules

SSR Deployment

- Node/server runtime concepts

79. CI/CD

Continuous Integration

- Installing dependencies
- Linting
- Testing
- Building

Continuous Deployment

- Development
- Staging
- Production

Common Platforms

- GitHub Actions
- Azure DevOps
- GitLab CI
- Jenkins

Deployment Quality Gates**80. Angular Development with AI Assistance****AI-Assisted Development**

- Generating component scaffolding
- Explaining Angular code
- Refactoring components
- Creating services
- Creating tests
- Debugging errors

Prompting for Angular

- Providing framework version
- Specifying standalone architecture
- Describing APIs and models
- Requesting typed TypeScript

AI Code Review

- Type safety
- RxJS cleanup
- Security
- Performance
- Accessibility

Responsible AI Usage

- Verify generated code
- Avoid outdated Angular patterns
- Validate package APIs
- Protect confidential code

81. Mini Projects

Students should complete multiple hands-on projects.

Mini Project 1 – Task Manager

Concepts:

- Components
- Data binding
- Events
- Signals
- Forms

Mini Project 2 – Expense Tracker

Concepts:

- Reactive forms
- Services
- Shared state
- Filtering

Mini Project 3 – Weather Application

Concepts:

- HTTP client
- API integration
- Loading and error states
- RxJS

Mini Project 4 – Product Catalog

Concepts:

- Routing
- Route parameters
- Search
- Filters
- API calls

Mini Project 5 – Employee Management System

Concepts:

- CRUD
- Reactive forms
- HTTP services
- Validation
- Routing

Mini Project 6 – Authentication Application

Concepts:

- Login
- Registration
- Guards
- Interceptors
- Token handling

Mini Project 7 – Admin Dashboard

Concepts:

- Angular Material
- Tables
- Charts
- Lazy loading
- State management
- Responsive design

82. Major Project / Capstone Project

Students should build at least one production-style Angular application.

Suggested Capstone 1 – E-Commerce Application

Features:

- Product catalog
- Product details
- Search
- Filtering
- Cart
- Authentication
- Checkout UI
- Orders
- Admin panel
- Role-based authorization

Suggested Capstone 2 – Learning Management System

Features:

- Student authentication
- Course catalog
- Enrollment
- Progress tracking
- Instructor dashboard
- Assessments
- Admin dashboard

Suggested Capstone 3 – HR Management System

Features:

- Employees
- Departments
- Attendance
- Leave requests
- Role-based access
- Reports
- Admin dashboard

Suggested Capstone 4 – Project Management Application

Features:

- Projects
- Tasks
- Teams
- Kanban board
- Comments
- Notifications
- Dashboard
- Permissions

Suggested Capstone 5 – Banking/Finance Dashboard

Features:

- Authentication
- Account summaries

- Transactions
- Filters
- Charts
- Statements
- Secure route handling

83. Capstone Project Requirements

The final application should demonstrate:

- Modern standalone Angular architecture
- TypeScript
- Reusable components
- Signals
- RxJS
- Routing
- Lazy loading
- Guards
- Reactive or signal-based forms
- Form validation
- REST API integration
- HTTP interceptors
- Authentication
- Authorization
- State management
- Loading and error handling
- Responsive UI
- Accessibility
- Unit testing
- Route/API testing
- Git/GitHub usage
- Production deployment
- Project documentation
- Professional README

84. Interview Preparation

Angular Interview Topics

- Angular architecture
- Components
- Templates
- Data binding
- Directives
- Pipes
- Dependency injection
- Services
- Signals
- Lifecycle hooks
- RxJS
- Observables
- Subjects
- Forms
- Routing
- Guards

- Lazy loading
- HTTP client
- Interceptors
- State management
- NgRx
- Change detection
- OnPush
- Standalone components
- SSR
- Hydration
- Testing

TypeScript Interview Topics

- Interfaces
- Type aliases
- Generics
- Classes
- Access modifiers
- Utility types
- Union types
- Type guards
- Strict typing

RxJS Interview Topics

- Observable vs Promise
- Subject vs BehaviorSubject
- switchMap
- mergeMap
- concatMap
- exhaustMap
- combineLatest
- forkJoin
- catchError
- Subscription cleanup

85. Practical Interview Exercises

Learners should be able to:

- Build a standalone Angular component
- Pass data between components
- Build a reactive form
- Create custom validation
- Consume a REST API
- Create an HTTP interceptor
- Build protected routes
- Implement lazy loading
- Implement search with debounce
- Use RxJS cancellation
- Build state using signals
- Explain NgRx architecture
- Optimize change detection
- Write component tests
- Debug a dependency injection error

86. Coding Exercises

Students should practice:

- Counter using signals
- Todo list
- Accordion
- Tabs
- Modal
- Dropdown
- Search box
- Debounced API search
- Pagination
- Infinite scrolling
- Dynamic forms
- Custom validators
- Shopping cart
- Theme switcher
- Authentication flow
- Protected routes
- CRUD application
- Data table
- Route parameter handling
- File upload UI
- Dashboard widgets

87. Recommended Modern Angular Stack

A modern Angular development stack may include:

- Angular 22.x
- TypeScript
- Angular CLI
- Standalone Components
- Angular Signals
- RxJS
- Angular Router
- Angular Forms
- Angular HttpClient
- Angular Material
- NgRx or signal-based state
- Vitest
- Angular Testing Utilities
- Playwright or Cypress
- ESLint
- Prettier
- Git
- GitHub
- Azure/AWS/Firebase deployment

88. Suggested Course Sequence

Phase 1 – Web and TypeScript Foundation

- HTML refresher
- CSS refresher

- JavaScript refresher
- TypeScript fundamentals
- Advanced TypeScript

Phase 2 – Angular Fundamentals

- Angular introduction
- Angular CLI
- Project structure
- Components
- Templates
- Data binding
- Inputs and outputs
- Directives
- Pipes
- Modern control flow

Phase 3 – Angular Reactivity

- Signals
- Computed state
- Effects
- RxJS fundamentals
- RxJS operators
- Signals and RxJS interoperability

Phase 4 – Forms and Application Features

- Template-driven forms
- Reactive forms
- Validation
- Signal forms
- Services
- Dependency injection

Phase 5 – Routing and APIs

- Angular Router
- Route parameters
- Nested routes
- Guards
- Lazy loading
- HttpClient
- Interceptors
- Authentication
- Authorization

Phase 6 – State and Architecture

- Services + signals
- RxJS state
- NgRx
- Application architecture
- Standalone-first architecture
- NgModules for legacy applications

Phase 7 – Production Angular

- Change detection
- Performance

- Testing
- Accessibility
- Security
- SSR
- Hydration
- SEO
- Deployment
- CI/CD

Phase 8 – Professional Development

- Git and GitHub
- Code quality
- Debugging
- AI-assisted Angular development

Phase 9 – Career Preparation

- Mini projects
- Capstone project
- GitHub portfolio
- Interview preparation
- Mock interviews

89. Learning Outcomes

After completing this Angular course, learners should be able to:

- Build modern SPA applications using Angular
- Write strongly typed TypeScript
- Create standalone Angular components
- Build reusable components, directives, and pipes
- Use Angular signals for reactive state
- Work effectively with RxJS
- Build template-driven, reactive, and signal-oriented forms
- Implement form validation
- Build scalable route structures
- Implement lazy loading
- Consume REST APIs
- Use HTTP interceptors
- Implement authentication and authorization
- Manage application state
- Use NgRx when appropriate
- Optimize Angular change detection
- Build accessible and responsive interfaces
- Write unit and integration tests
- Build SSR-capable Angular applications
- Understand hydration
- Structure enterprise Angular applications
- Use Git and GitHub professionally
- Deploy Angular applications
- Build portfolio-ready projects
- Prepare for Angular developer interviews

90. Suggested Course Deliverables

By the end of the course, each learner should ideally have:

- TypeScript coding assignments
- Angular component exercises
- RxJS exercises
- Form-validation assignments
- API-integration assignments
- 5–7 mini projects
- At least one major capstone project
- GitHub repositories
- Professional project README files
- Deployed Angular applications
- Angular interview preparation
- TypeScript interview preparation
- RxJS interview preparation
- Mock interview experience
- AI-assisted development practice
- A frontend development portfolio

91. Optional Advanced Modules

Angular CDK

- Overlay
- Portal concepts
- Drag and drop
- Virtual scrolling
- Accessibility helpers

Advanced NgRx

- Entity
- Router Store
- Component Store concepts
- Signal Store concepts
- Effects architecture

Advanced RxJS

- Custom operators
- Multicasting
- Schedulers concepts
- Advanced error recovery

Micro Frontends

- Module Federation
- Independent feature deployment
- Shared dependencies

Nx

- Enterprise monorepos
- Library boundaries
- Build caching
- Code ownership

Real-Time Applications

- WebSockets
- Socket.IO concepts
- Live notifications

- Real-time dashboards

GraphQL

- GraphQL fundamentals
- Queries
- Mutations
- Apollo Angular concepts

92. Career Roles After Angular Training

This course can help learners prepare for roles such as:

- Angular Developer
- Frontend Developer
- UI Developer
- TypeScript Developer
- Web Application Developer
- Frontend Software Engineer
- Angular Full-Stack Developer
- Enterprise UI Developer
- Senior Angular Developer

Conclusion

Angular is a complete frontend application framework used extensively for enterprise-grade web development. A job-ready Angular developer should know much more than components and templates—they should understand TypeScript, signals, RxJS, dependency injection, forms, routing, APIs, authentication, state management, testing, performance, accessibility, security, SSR, deployment, Git, and scalable application architecture.

This syllabus provides a structured learning path from foundational web development to modern production-ready Angular application development, while also preparing learners to maintain older enterprise applications that still use NgModules and other legacy Angular patterns.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.